

## Tarefa 2

### Paralelismo ao Nível de Instrução (ILP): Análise de Dependências e Otimizações de Compilador

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Maio de 2026

---

## 1. Introdução

O **Paralelismo ao Nível de Instrução** (ILP — *Instruction-Level Parallelism*) é a capacidade de um processador executar múltiplas instruções simultaneamente, desde que não existam dependências de dados entre elas. Compiladores modernos exploram o ILP por meio de técnicas como *pipeline*, execução fora de ordem e vetorização SIMD. O objetivo desta tarefa é investigar, de forma experimental, como diferentes estilos de codificação — e diferentes níveis de otimização do compilador GCC — afetam o desempenho de laços que operam sobre vetores de ponto flutuante de dupla precisão.

Foram implementados três laços em linguagem C, operando sobre um vetor estático de  $N = 32.000.000$  elementos do tipo `double`. Os laços foram compilados com os flags `-O0`, `-O2` e `-O3`, e os tempos de execução foram medidos via `clock_gettime(CLOCK_MONOTONIC)`.

## 2. Metodologia

### 2.1 Ambiente de Execução

Os experimentos foram executados em um sistema Linux (Ubuntu) em modo sequencial, sem paralelismo explícito de threads. O vetor `v[N]` foi declarado como variável global estática para evitar alocação dinâmica. A variável de resultado foi declarada `volatile` para impedir que o compilador elimine os laços por otimização de código morto.

### 2.2 Descrição dos Laços

#### Loop 1 — Inicialização do vetor (sem dependência entre iterações).

Cada elemento `v[i]` é calculado independentemente como  $i \times \pi + e$ . Não há dependência de dados entre iterações, tornando o laço altamente paralelizável no nível de instrução.

#### Loop 2 — Soma acumulativa com dependência verdadeira (RAW).

Um único acumulador `soma` é atualizado a cada iteração: `soma += v[i]`. Existe uma dependência *Read After Write* (RAW) entre iterações consecutivas — a iteração  $i+1$  depende do resultado de  $i$ . Isso cria uma cadeia de dependências que limita o ILP.

#### Loop 3 — Soma com quatro acumuladores independentes (quebra de dependência).

O laço é reformulado com quatro variáveis independentes  $s_0, s_1, s_2, s_3$ , cada uma acumulando elementos separados do vetor ( $v[i], v[i+1], v[i+2], v[i+3]$ ). As quatro cadeias de dependência são independentes entre si, permitindo execução paralela na unidade de ponto flutuante do processador.

## 3. Resultados

### 3.1 Tempos de Execução

A Tabela 1 apresenta os tempos de execução medidos (em segundos) para cada laço e nível de otimização.

Tabela 1 — Tempos de execução (em segundos)

| Loop                | -O0 (s)  | -O2 (s)  | -O3 (s)  |
|---------------------|----------|----------|----------|
| Loop 1 (init)       | 0.180915 | 0.137690 | 0.125349 |
| Loop 2 (soma dep)   | 0.089872 | 0.024289 | 0.023419 |
| Loop 3 (soma nodep) | 0.023742 | 0.013194 | 0.011090 |

### 3.2 Aceleração (Speedup)

A Tabela 2 resume os fatores de aceleração relativos entre os níveis de otimização e entre os Loops 2 e 3.

Tabela 2 — Fatores de aceleração

| Métrica             | O2 vs O0 | O3 vs O0 | Referência  |
|---------------------|----------|----------|-------------|
| Loop 1 (init)       | 1.31x    | 1.44x    | —           |
| Loop 2 (soma dep)   | 3.70x    | 3.84x    | —           |
| Loop 3 (soma nodep) | 1.80x    | 2.14x    | —           |
| Loop3 vs Loop2 (O0) | 3.79x    | —        | dependência |
| Loop3 vs Loop2 (O2) | —        | 1.84x    | dependência |
| Loop3 vs Loop2 (O3) | —        | 2.11x    | dependência |

## 4. Análise e Discussão

### 4.1 Loop 1 — Inicialização sem dependência

O Loop 1 é o mais intensivo em operações de escrita na memória, pois acessa todos os  $N$  elementos sequencialmente. A melhoria de -O0 para -O3 foi de aproximadamente **1,44x**. O ganho moderado reflete o fato de que o gargalo principal é a largura de banda de memória

(*memory-bound*), e não o cálculo aritmético. Com `-O3`, o compilador pode vetorizar o cálculo  $i \times \pi + e$  usando instruções SIMD (SSE/AVX), o que reduz o número de ciclos por iteração.

## 4.2 Loop 2 — Dependência RAW e seu impacto no ILP

O ganho de `-O0` para `-O2` foi de **3,70×**, o maior salto individual observado. Sob `-O0`, o compilador não realiza nenhuma otimização: cada instrução de soma de ponto flutuante aguarda a conclusão da anterior (latência típica de 4–5 ciclos em arquiteturas x86-64), resultando em tempo de execução elevado. Com `-O2` e `-O3`, o compilador reduz a sobrecarga de chamadas de função e melhora o escalonamento de instruções, mas a dependência RAW fundamental permanece — por isso a diferença entre `-O2` e `-O3` é pequena ( $\approx 3,5\%$ ).

## 4.3 Loop 3 — Quebra de dependência e ganho de ILP

A técnica de usar múltiplos acumuladores é conhecida como *loop unrolling with accumulator splitting*. Ao substituir uma única cadeia de dependência por quatro cadeias independentes, o processador pode manter as quatro operações de soma em diferentes estágios do *pipeline* simultaneamente. O resultado é expressivo: o Loop 3 executou **3,78×** mais rápido que o Loop 2 com `-O0` e **2,11×** mais rápido com `-O3`.

O fato de o ganho ser menor em `-O3` indica que o compilador já realiza, automaticamente, algum grau de vetorização e desdobramento de laço no Loop 2, reduzindo parcialmente o efeito das dependências. Ainda assim, o Loop 3 permanece mais rápido, evidenciando que a reformulação manual do código é eficaz mesmo quando o compilador já otimiza.

## 4.4 Diferença numérica nos resultados

Os dois laços de soma produzem resultados ligeiramente diferentes (1608495473519536.00 vs 1608495473519535.75). Isso é esperado: a aritmética de ponto flutuante *não é associativa*. Ao alterar a ordem de operações, a acumulação de erros de arredondamento pode diferir em até algumas ULPs (*Units in the Last Place*). Para a maioria das aplicações científicas, essa diferença é desprezível.

## 5. Conclusão

Os experimentos demonstraram, de forma clara, a influência de dois fatores sobre o desempenho de laços numéricos:

**(i) Dependências de dados:** a dependência RAW no Loop 2 limita severamente o ILP e é o principal obstáculo ao desempenho, especialmente sem otimização do compilador. A reformulação manual com múltiplos acumuladores (Loop 3) reduziu o tempo em até 3,78× em relação ao Loop 2.

**(ii) Nível de otimização do compilador:** os flags `-O2` e `-O3` trouxeram ganhos significativos, especialmente em código com menos otimização manual. O Loop 2, com dependência explícita, foi o que mais se beneficiou da compilação otimizada (3,70×). Já o Loop 3 apresentou ganho mais modesto (2,14×), pois o próprio código já expõe o ILP ao compilador.

Conclui-se que a escrita de código que expõe o ILP explicitamente, aliada ao uso de flags de otimização adequados, é fundamental para extrair o máximo de desempenho de processadores modernos — mesmo em aplicações sequenciais, sem paralelismo de threads.

## 6. Código Fonte

```
1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <time.h>
4
5  #define N      32000000
6  #define BILLION 1000000000.0
7
8  static double v[N];
9
10 static double elapsed(struct timespec a, struct timespec b) {
11     return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / BILLION;
12 }
13
14 /* Loop 1: inicializa vetor – sem dependência entre iterações (ILP-friendly) */
15 void loop_init(void) {
16     for (int i = 0; i < N; i++)
17         v[i] = (double)i * 3.14159265 + 2.71828182;
18 }
19
20 /* Loop 2: soma acumulativa – dependência verdadeira (RAW) entre iterações */
21 double loop_soma_dep(void) {
22     double soma = 0.0;
23     for (int i = 0; i < N; i++)
24         soma += v[i];
25     return soma;
26 }
27
28 /* Loop 3: quebra a dependência com 4 acumuladores independentes */
29 double loop_soma_nodep(void) {
30     double s0 = 0.0, s1 = 0.0, s2 = 0.0, s3 = 0.0;
31     for (int i = 0; i < N; i += 4) {
32         s0 += v[i];
33         s1 += v[i + 1];
34         s2 += v[i + 2];
35         s3 += v[i + 3];
36     }
37     return s0 + s1 + s2 + s3;
38 }
39
40 int main(void) {
41     struct timespec ini, fim;
42     volatile double res;
43     double t;
44
45     clock_gettime(CLOCK_MONOTONIC, &ini);
46     loop_init();
47     clock_gettime(CLOCK_MONOTONIC, &fim);
48     t = elapsed(ini, fim);
49     printf("Loop 1 (init):      %.6f s\n", t);
50
51     clock_gettime(CLOCK_MONOTONIC, &ini);
52     res = loop_soma_dep();
53     clock_gettime(CLOCK_MONOTONIC, &fim);
54     t = elapsed(ini, fim);
55     printf("Loop 2 (soma dep):   %.6f s  (res = %.2f)\n", t, res);
56
57     clock_gettime(CLOCK_MONOTONIC, &ini);
58     res = loop_soma_nodep();
59     clock_gettime(CLOCK_MONOTONIC, &fim);
60     t = elapsed(ini, fim);
61     printf("Loop 3 (soma nodep):  %.6f s  (res = %.2f)\n", t, res);
62
63     return 0;
64 }
65
```